

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

Amitrakshar Mukherjee (1BM24CS035)

**in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING**



**B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026**

CERTIFICATE

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering**



This is to certify that the Lab work entitled “**ANALYSIS AND DESIGN OF ALGORITHMS**” carried out by **Amitrakshar Mukherjee (1BM24CS035)**, who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - **(23CS4PCADA)** work prescribed for the said degree.

Dr. Kanchana M Dixit
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1	Write program to obtain the Topological ordering of vertices in a given digraph.	1-3
2	Implement Johnson Trotter algorithm to generate permutations.	4-7
3	Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.	8-10
4	Sort a given set of N integer elements using Quick Sort technique and compute its time taken.	11-14
5	Sort a given set of N integer elements using Heap Sort technique and compute its time taken.	15-18
6	Implement 0/1 Knapsack problem using dynamic programming. (5 Marks)	19-21
7	Implement All Pair Shortest paths problem using Floyd's algorithm.	22-24
8	(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm. (b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	25-32
9	Implement Fractional Knapsack using Greedy technique.	33-35
10	From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.	36-38
11	Implement "N-Queens Problem" using Backtracking.	39-42

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Lab program 1:

Write program to obtain the Topological ordering of vertices in a given digraph.

```
#include <stdio.h>

#define MAX 10

int graph[MAX][MAX], visited[MAX], stack[MAX];
int n, top = -1;

void dfs(int v)
{
    visited[v] = 1;

    for(int i = 0; i < n; i++)
        if(graph[v][i] && !visited[i])
            dfs(i);

    stack[++top] = v;
}

void topologicalSort()
{
    for(int i = 0; i < n; i++)
        if(!visited[i])
            dfs(i);

    printf("\nTopological Ordering: ");
    while(top != -1)
        printf("%d ", stack[top--]);
}
```

```
    printf("\n");
}

int main()
{
    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix:\n");

    for(int i = 0; i < n; i++)
    {
        visited[i] = 0;
        for(int j = 0; j < n; j++)
            scanf("%d", &graph[i][j]);
    }

    topologicalSort();

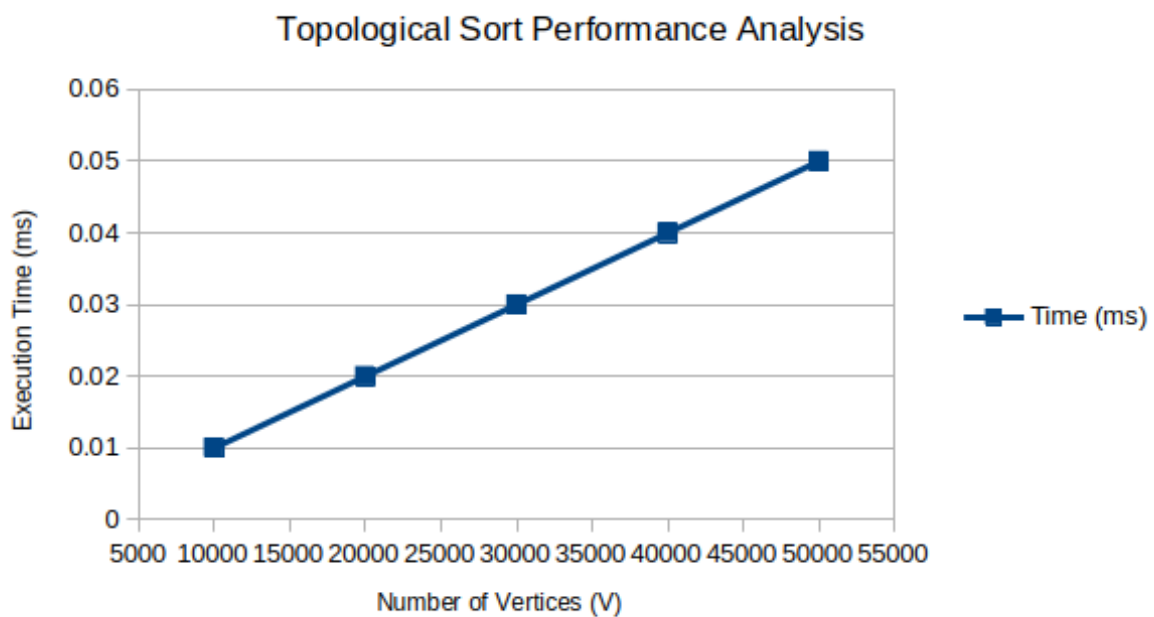
    return 0;
}
```

OUTPUT

```
/home/aurko/Downloads/adalab
Enter number of vertices: 6
Enter adjacency matrix:
0 0 1 1 0 0
0 0 0 1 1 0
0 0 0 0 0 1
0 0 0 0 0 1
0 0 0 0 0 1
0 0 0 0 0 0

Topological Ordering: 1 4 0 3 2 5
Process returned 0 (0x0)   execution time : 29.992 s
Press ENTER to continue.
█
```

Graph



Lab program 2:

Implement Johnson Trotter algorithm to generate permutations.

```
#include<stdio.h>

#define SIZE 5

int a[SIZE],max,pos,n;
char dir[SIZE];

void print()
{
    for(int i=0;i<n;i++)printf("%d(%c) ",a[i],dir[i]);
    printf("\n");
}

int main()
{
    printf("Enter n value: ");
    scanf("%d",&n);

    for(int i=0;i<n;i++)
    {
        a[i]=i+1;
        dir[i]='L';
    }

    print();

    while(1)
    {
```

```

max=-1;
pos=-1;

for(int i=0;i<n;i++)
    if((dir[i]=='L'&& i>0&&a[i]>a[i-1])||(dir[i]=='R'&&i<n-1&&a[i]>a[i+1]))
        if(a[i]>max)
        {
            max=a[i];
            pos=i;
        }

if(max== -1)break;

if(dir[pos]=='L')
{
    int temp=a[pos];
    a[pos]=a[pos-1];
    a[pos-1]=temp;

    char tempdir=dir[pos];
    dir[pos]=dir[pos-1];
    dir[pos-1]=tempdir;
}
else
{
    int temp=a[pos];
    a[pos]=a[pos+1];
    a[pos+1]=temp;

```

```
        char tempdir=dir[pos];
        dir[pos]=dir[pos+1];
        dir[pos+1]=tempdir;
    }

    for(int i=0;i<n;i++)
        if(a[i]>max)
            dir[i]=(dir[i]=='L')?'R':'L';

    print();
}

return 0;
}
```

OUTPUT

```
/home/aurko/Downloads/adalab
Enter n value: 4
1(L) 2(L) 3(L) 4(L)
1(L) 2(L) 4(L) 3(L)
1(L) 4(L) 2(L) 3(L)
4(L) 1(L) 2(L) 3(L)
4(R) 1(L) 3(L) 2(L)
1(L) 4(R) 3(L) 2(L)
1(L) 3(L) 4(R) 2(L)
1(L) 3(L) 2(L) 4(R)
3(L) 1(L) 2(L) 4(L)
3(L) 1(L) 4(L) 2(L)
3(L) 4(L) 1(L) 2(L)
4(L) 3(L) 1(L) 2(L)
4(R) 3(R) 2(L) 1(L)
3(R) 4(R) 2(L) 1(L)
3(R) 2(L) 4(R) 1(L)
3(R) 2(L) 1(L) 4(R)
2(L) 3(R) 1(L) 4(L)
2(L) 3(R) 4(L) 1(L)
2(L) 4(L) 3(R) 1(L)
4(L) 2(L) 3(R) 1(L)
4(R) 2(L) 1(L) 3(R)
2(L) 4(R) 1(L) 3(R)
2(L) 1(L) 4(R) 3(R)
2(L) 1(L) 3(R) 4(R)

Process returned 0 (0x0)   execution time : 25.108 s
Press ENTER to continue.
█
```

Lab program 3:

Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.

```
#include<stdio.h>

#include<time.h>


int main()
{
    int low,mid,high;


    printf("Enter low: ");
    scanf("%d",&low);


    printf("Enter mid: ");
    scanf("%d",&mid);


    printf("Enter high: ");
    scanf("%d",&high);


    int a[100],b[100];


    printf("Enter %d sorted elements:\n",(high-low)+1);


    for(int i=low;i<=high;i++)
        scanf("%d",&a[i]);


    clock_t start,end;
    double cpu_time_used;
```

```
start=clock();

int i=low,j=mid+1,k=low;

while(i<=mid&& j<=high)
{
    if(a[i]<=a[j])
        b[k++]=a[i++];
    else
        b[k++]=a[j++];
}

while(i<=mid)
    b[k++]=a[i++];

while(j<=high)
    b[k++]=a[j++];

end=clock();

cpu_time_used=((double)(end-start))/CLOCKS_PER_SEC;

printf("Merged array:\n");

for(k=low;k<=high;k++)
    printf("%d ",b[k]);

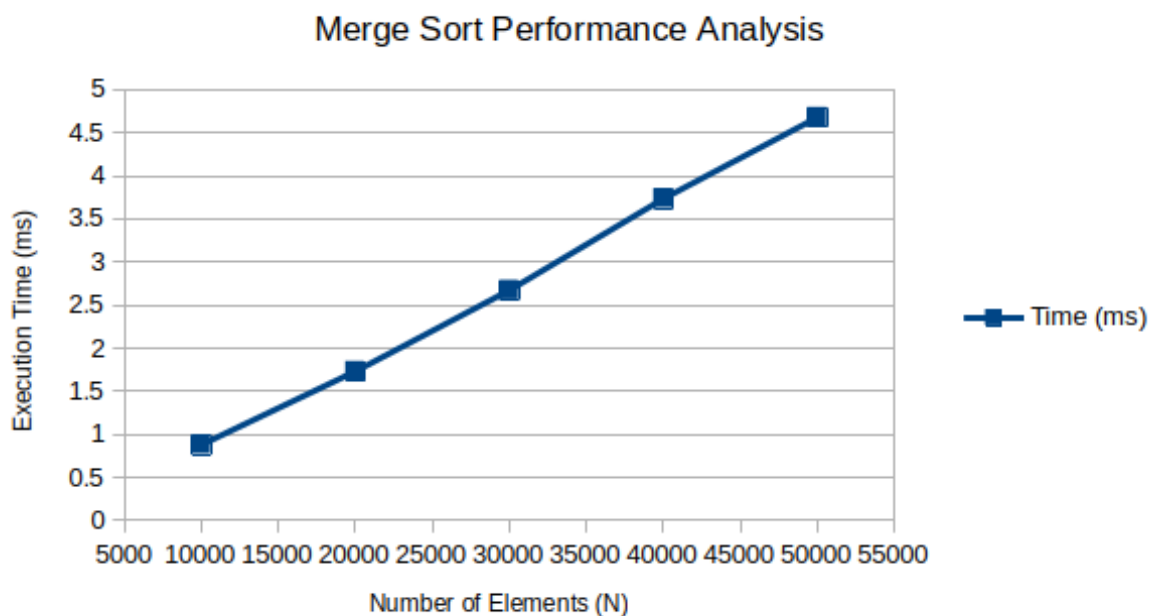
printf("\nExecution time: %f seconds\n",cpu_time_used);
```

```
    return 0;  
}
```

OUTPUT

```
/home/aurko/Downloads/adalab  
Enter low: 0  
Enter mid: 2  
Enter high: 5  
Enter 6 sorted elements:  
10 30 50 20 40 60  
Merged array:  
10 20 30 40 50 60  
Execution time: 0.000004 seconds  
  
Process returned 0 (0x0)   execution time : 7.644 s  
Press ENTER to continue.  
█
```

Graph



Lab program 4:

Sort a given set of N integer elements using Quick Sort technique and compute its time taken.

```
#include<stdio.h>
```

```
#include<time.h>
```

```
int partition(int a[100],int low,int high)
```

```
{
```

```
    int i,j,key,temp;
```

```
    i=low+1;
```

```
    j=high;
```

```
    key=a[low];
```

```
    while(1)
```

```
    {
```

```
        while(a[i]<=key&& i<high)i++;
```

```
        while(a[j]>key)j--;
```

```
        if(i<j)
```

```
        {
```

```
            temp=a[i];
```

```
            a[i]=a[j];
```

```
            a[j]=temp;
```

```
        }
```

```
    else
```

```
    {
```

```
        temp=a[low];
```

```
        a[low]=a[j];
```

```

        a[j]=temp;
        return j;
    }
}

void quicksort(int a[100],int low,int high)
{
    int j;

    if(low<high)
    {
        j=partition(a,low,high);
        quicksort(a,low,j-1);
        quicksort(a,j+1,high);
    }
}

int main()
{
    int low,high,i;
    clock_t start,end;
    double time_taken;

    printf("Enter low: ");
    scanf("%d",&low);

    printf("Enter high: ");
    scanf("%d",&high);

```

```
int a[100];

printf("Enter %d elements:\n",(high-low)+1);

for(i=low;i<=high;i++)
    scanf("%d",&a[i]);

start=clock();

quicksort(a,low,high);

end=clock();

printf("Sorted array:\n");

for(i=low;i<=high;i++)
    printf("%d ",a[i]);

time_taken=(double)(end-start)/CLOCKS_PER_SEC;

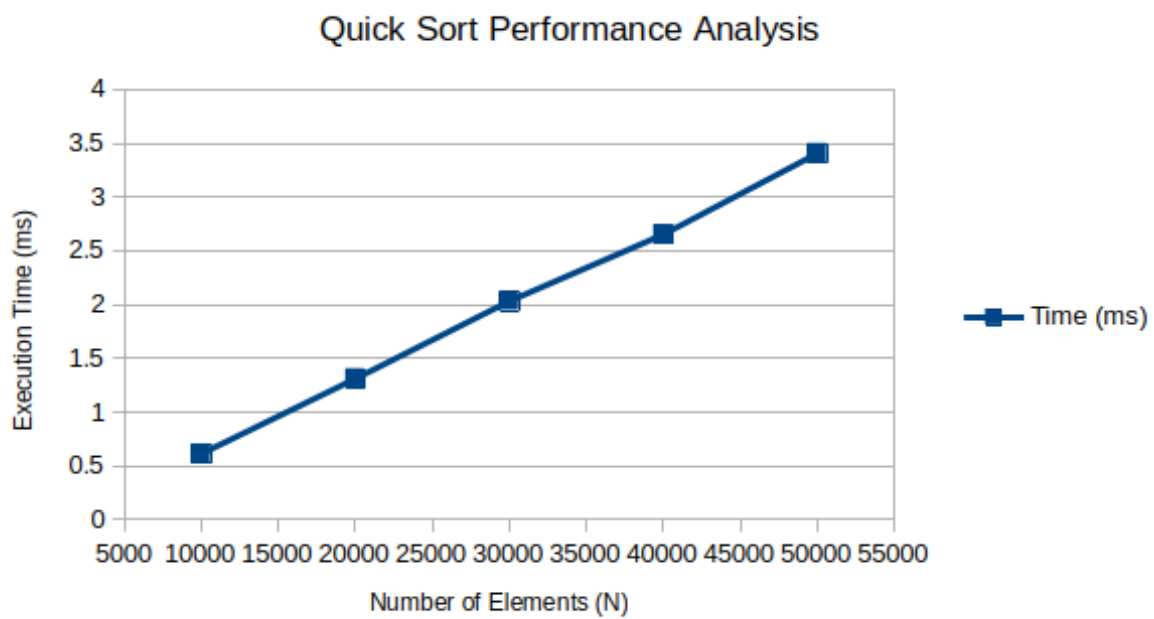
printf("\nTime taken = %f seconds",time_taken);

return 0;
}
```

OUTPUT

```
/home/aurko/Downloads/adalab
Enter low: 0
Enter high: 5
Enter 6 elements:
50 30 70 10 40 20
Sorted array:
10 20 30 40 50 70
Time taken = 0.000002 seconds
Process returned 0 (0x0)   execution time : 15.999 s
Press ENTER to continue.
```

Graph



Lab program 5:

Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

```
#include<stdio.h>
```

```
#include<time.h>
```

```
void heapify(int a[],int n,int i)
```

```
{
```

```
    int largest=i;
```

```
    int left=2*i+1;
```

```
    int right=2*i+2;
```

```
    int temp;
```

```
    if(left<n&& a[left]>a[largest])
```

```
        largest=left;
```

```
    if(right<n&& a[right]>a[largest])
```

```
        largest=right;
```

```
    if(largest!=i)
```

```
    {
```

```
        temp=a[i];
```

```
        a[i]=a[largest];
```

```
        a[largest]=temp;
```

```
        heapify(a,n,largest);
```

```
    }
```

```
}
```

```

void heapSort(int a[],int n)
{
    int i,temp;

    for(i=n/2-1;i>=0;i--)
        heapify(a,n,i);

    for(i=n-1;i>0;i--)
    {
        temp=a[0];
        a[0]=a[i];
        a[i]=temp;

        heapify(a,i,0);
    }
}

int main()
{
    int a[1000],n,i;
    clock_t start,end;
    double time_taken;

    printf("Enter number of elements: ");
    scanf("%d",&n);

    printf("Enter elements:\n");

    for(i=0;i<n;i++)

```

```
        scanf("%d",&a[i]);

start=clock();

heapSort(a,n);

end=clock();

printf("\nSorted array:\n");

for(i=0;i<n;i++)
    printf("%d ",a[i]);

time_taken=((double)(end-start))/CLOCKS_PER_SEC;

printf("\n\nTime taken = %f seconds\n",time_taken);

return 0;
}
```

OUTPUT

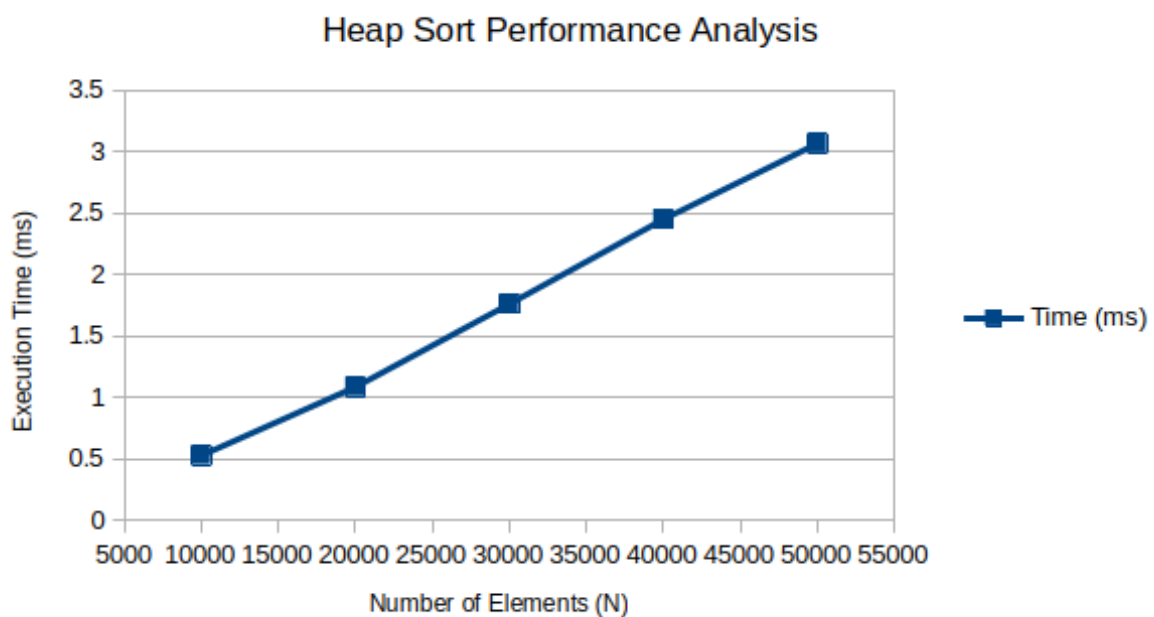
```
/home/aurko/Downloads/adalab
Enter number of elements: 6
Enter elements:
50 30 70 10 40 20

Sorted array:
10 20 30 40 50 70

Time taken = 0.000005 seconds

Process returned 0 (0x0)   execution time : 16.041 s
Press ENTER to continue.
```

Graph



Lab Program 6:-

Implement 0/1 Knapsack problem using dynamic programming

```
#include<stdio.h>
```

```
int max(int a,int b)
```

```
{
```

```
    if(a>b)
```

```
        return a;
```

```
    else
```

```
        return b;
```

```
}
```

```
int knapsack(int n,int m,int w[],int p[])
```

```
{
```

```
    int i,j;
```

```
    int table[n+1][m+1];
```

```
    for(i=0;i<=n;i++)
```

```
        table[i][0]=0;
```

```
    for(j=0;j<=m;j++)
```

```
        table[0][j]=0;
```

```
    for(i=1;i<=n;i++)
```

```
    {
```

```
        for(j=1;j<=m;j++)
```

```
        {
```

```
            if(j<w[i-1])
```

```
                table[i][j]=table[i-1][j];
```

```

        else
            table[i][j]=max(table[i-1][j],p[i-1]+table[i-1][j-w[i-1]]);
        }
    }

    return table[n][m];
}

int main()
{
    int n,m;

    printf("Enter number of items: ");
    scanf("%d",&n);

    int w[n],p[n];

    printf("Enter weights of items:\n");
    for(int i=0;i<n;i++)
        scanf("%d",&w[i]);

    printf("Enter profits of items:\n");
    for(int i=0;i<n;i++)
        scanf("%d",&p[i]);

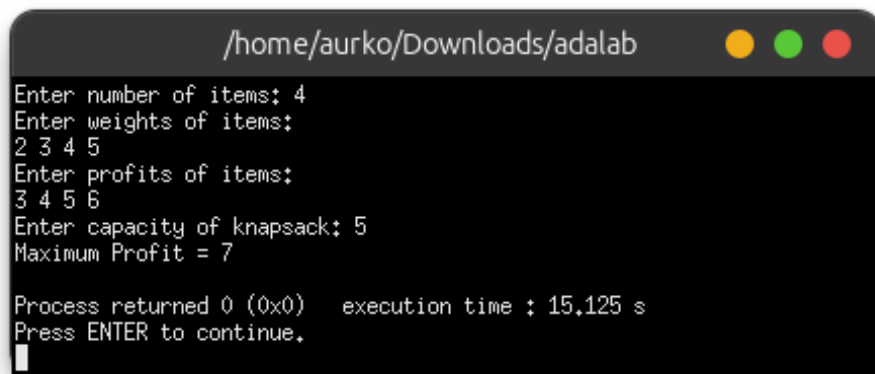
    printf("Enter capacity of knapsack: ");
    scanf("%d",&m);

    printf("Maximum Profit = %d\n",knapsack(n,m,w,p));

```

```
    return 0;  
}
```

OUTPUT

A terminal window with a dark background and light gray text. The title bar at the top shows the path "/home/aurko/Downloads/adalab" and three window control buttons (yellow, green, red). The terminal content shows a series of prompts and user input for a knapsack problem. The prompts are "Enter number of items:", "Enter weights of items:", "Enter profits of items:", and "Enter capacity of knapsack:". The user input is "4", "2 3 4 5", "3 4 5 6", and "5" respectively. The program outputs "Maximum Profit = 7". At the bottom, it shows "Process returned 0 (0x0) execution time : 15.125 s" and "Press ENTER to continue." with a cursor on the next line.

```
/home/aurko/Downloads/adalab  
Enter number of items: 4  
Enter weights of items:  
2 3 4 5  
Enter profits of items:  
3 4 5 6  
Enter capacity of knapsack: 5  
Maximum Profit = 7  
  
Process returned 0 (0x0)   execution time : 15.125 s  
Press ENTER to continue.  
█
```

Lab Program 7:-

Implement All Pair Shortest paths problem using Floyd's algorithm.

```
#include<stdio.h>

#include<time.h>

#define MAX 20
#define INF 9999

void floyds(int n,int cost[MAX][MAX],int A[MAX][MAX])
{
    int i,j,k;

    for(i=0;i<n;i++)
        for(j=0;j<n;j++)
            A[i][j]=cost[i][j];

    for(k=0;k<n;k++)
        for(i=0;i<n;i++)
            for(j=0;j<n;j++)
                if(A[i][k]+A[k][j]<A[i][j])
                    A[i][j]=A[i][k]+A[k][j];
}

int main()
{
    int n;

    int cost[MAX][MAX],A[MAX][MAX];

    int i,j;

    clock_t start,end;
```

```

double cpu_time_used;

printf("Enter number of vertices: ");
scanf("%d",&n);

printf("Enter the cost matrix:\n");
printf("(Enter %d for INF / no direct edge)\n",INF);

for(i=0;i<n;i++)
    for(j=0;j<n;j++)
        scanf("%d",&cost[i][j]);

start=clock();

floyds(n,cost,A);

end=clock();

cpu_time_used=((double)(end-start))/CLOCKS_PER_SEC;

printf("\nShortest Distance Matrix:\n");

for(i=0;i<n;i++)
{
    for(j=0;j<n;j++)
    {
        if(A[i][j]==INF)
            printf("INF\t");
        else

```

```

        printf("%d\t",A[i][j]);
    }
    printf("\n");
}

printf("\nExecution Time: %f seconds\n",cpu_time_used);

return 0;
}

```

OUTPUT

```

/home/aurko/Downloads/adalab
Enter number of vertices: 4
Enter the cost matrix:
(Enter 9999 for INF / no direct edge)
0 5 9999 10
9999 0 3 9999
9999 9999 0 1
9999 9999 9999 0

Shortest Distance Matrix:
0      5      8      9
INF    0      3      4
INF    INF    0      1
INF    INF    INF    0

Execution Time: 0.000006 seconds

Process returned 0 (0x0)   execution time : 35.020 s
Press ENTER to continue.

```

Lab Program 8:-

(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.

```
#include<stdio.h>

#define INF 9999
#define MAX 10

void prims(int n,int cost[MAX][MAX])
{
    int near[MAX],t[MAX][2];
    int i,j,k,l,min,mincost;

    min=INF;

    for(i=1;i<=n;i++)
        for(j=i+1;j<=n;j++)
            if(cost[i][j]<min)
            {
                min=cost[i][j];
                k=i;
                l=j;
            }

    t[1][0]=k;
    t[1][1]=l;
    mincost=cost[k][l];

    for(i=1;i<=n;i++)
```

```

{
    if(cost[i][k]<cost[i][l])
        near[i]=k;
    else
        near[i]=l;
}

near[k]=0;
near[l]=0;

for(i=2;i<=n-1;i++)
{
    min=INF;

    for(j=1;j<=n;j++)
        if(near[j]!=0&&cost[j][near[j]]<min)
        {
            min=cost[j][near[j]];
            k=j;
        }

    t[i][0]=k;
    t[i][1]=near[k];

    mincost+=cost[k][near[k]];
    near[k]=0;

    for(j=1;j<=n;j++)
        if(near[j]!=0&&cost[j][near[j]]>cost[j][k])

```

```

        near[j]=k;
    }

    printf("\nEdges in MST:\n");

    for(i=1;i<=n-1;i++)
        printf("%d - %d\n",t[i][0],t[i][1]);

    printf("Minimum Cost = %d\n",mincost);
}

int main()
{
    int n,cost[MAX][MAX];
    int i,j;

    printf("Enter number of vertices: ");
    scanf("%d",&n);

    printf("Enter cost matrix:\n");

    for(i=1;i<=n;i++)
        for(j=1;j<=n;j++)
        {
            scanf("%d",&cost[i][j]);

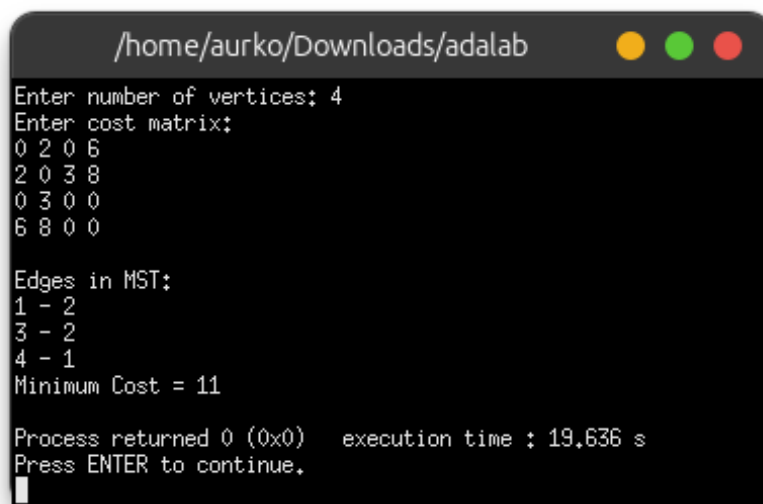
            if(cost[i][j]==0)
                cost[i][j]=INF;
        }
}

```

```
prims(n,cost);

return 0;
}
```

OUTPUT



```
/home/aurko/Downloads/adalab
Enter number of vertices: 4
Enter cost matrix:
0 2 0 6
2 0 3 8
0 3 0 0
6 8 0 0

Edges in MST:
1 - 2
3 - 2
4 - 1
Minimum Cost = 11

Process returned 0 (0x0)   execution time : 19.636 s
Press ENTER to continue.
```

(b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

```
#include<stdio.h>

#define MAX 10
#define INF 9999

int parent[MAX];

int find(int i)
{
    while(parent[i])
        i=parent[i];
    return i;
}

void union_set(int i,int j)
{
    parent[j]=i;
}

int main()
{
    int n,cost[MAX][MAX];
    int t[MAX][2];
    int i,j,u,v;
    int min,mincost=0;
    int edge_count=0;
```

```
printf("Enter number of vertices: ");  
scanf("%d",&n);
```

```
printf("Enter cost matrix:\n");
```

```
for(i=1;i<=n;i++)  
    for(j=1;j<=n;j++)  
    {  
        scanf("%d",&cost[i][j]);  
  
        if(cost[i][j]==0)  
            cost[i][j]=INF;  
    }
```

```
for(i=1;i<=n;i++)  
    parent[i]=0;
```

```
while(edge_count<n-1)  
{  
    min=INF;
```

```
    for(i=1;i<=n;i++)  
        for(j=1;j<=n;j++)  
            if(cost[i][j]<min)  
            {  
                min=cost[i][j];  
                u=i;  
                v=j;  
            }
```

```

i=find(u);
j=find(v);

if(i!=j)
{
    edge_count++;

    t[edge_count][0]=u;
    t[edge_count][1]=v;

    mincost+=min;

    union_set(i,j);
}

cost[u][v]=cost[v][u]=INF;
}

printf("\nEdges in MST:\n");

for(i=1;i<=edge_count;i++)
    printf("%d - %d\n",t[i][0],t[i][1]);

printf("Minimum Cost = %d\n",mincost);

return 0;
}

```

OUTPUT

A terminal window with a dark background and light gray text. The title bar at the top shows the path "/home/aurko/Downloads/adalab" and three window control buttons (yellow, green, red). The terminal content shows a program that takes input for the number of vertices and a cost matrix, then outputs the edges of a Minimum Spanning Tree (MST) and the minimum cost. The input is 4 vertices and a 4x4 cost matrix. The output shows three edges: 1-2, 2-3, and 1-4, with a minimum cost of 11. At the bottom, it says "Process returned 0 (0x0) execution time : 16.775 s" and "Press ENTER to continue." with a cursor on the next line.

```
/home/aurko/Downloads/adalab
Enter number of vertices: 4
Enter cost matrix:
0 2 0 6
2 0 3 8
0 3 0 0
6 8 0 0

Edges in MST:
1 - 2
2 - 3
1 - 4
Minimum Cost = 11

Process returned 0 (0x0)   execution time : 16.775 s
Press ENTER to continue.
█
```

Lab Program 9:-

Implement Fractional Knapsack using Greedy technique.

```
#include<stdio.h>

struct item
{
    float p;
    float w;
    float r;
    float x;
};

int main()
{
    int n;
    float m;

    printf("Enter the number of items: ");
    scanf("%d",&n);

    printf("Enter max weight: ");
    scanf("%f",&m);

    struct item temp;
    struct item items[50];

    for(int i=0;i<n;i++)
    {
        printf("Enter weight of item %d: ",i+1);
```

```

scanf("%f",&items[i].w);

printf("Enter profit of item %d: ",i+1);
scanf("%f",&items[i].p);

items[i].r=items[i].p/items[i].w;
items[i].x=0;
}

for(int i=0;i<n;i++)
{
    int max=i;

    for(int j=i+1;j<n;j++)
        if(items[j].r>items[max].r)
            max=j;

    temp=items[i];
    items[i]=items[max];
    items[max]=temp;
}

float remain=m;
float maxprofit=0;
int i=0;

while(i<n)
{
    if(items[i].w>remain)

```

```

        break;

    items[i].x=1;
    remain-=items[i].w;
    maxprofit+=items[i].p;
    i++;
}

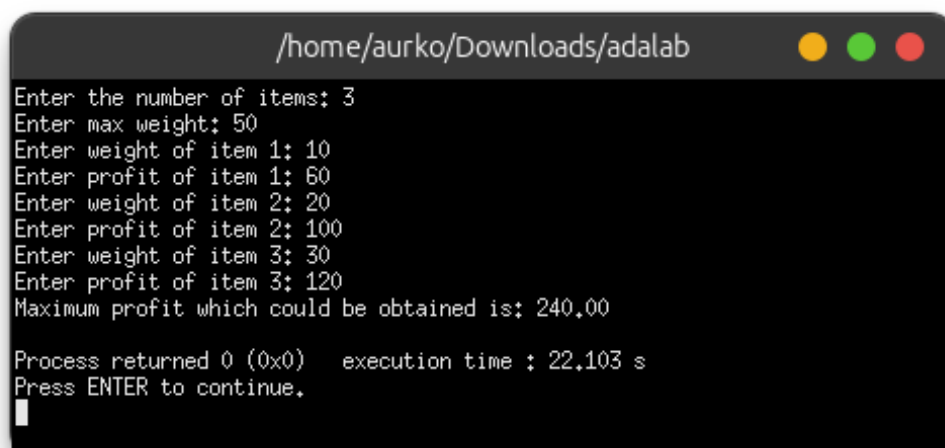
if(i<n)
{
    items[i].x=remain/items[i].w;
    maxprofit+=items[i].x*items[i].p;
}

printf("Maximum profit which could be obtained is: %.2f\n",maxprofit);

return 0;
}

```

OUTPUT



```

/home/aurko/Downloads/adalah
Enter the number of items: 3
Enter max weight: 50
Enter weight of item 1: 10
Enter profit of item 1: 60
Enter weight of item 2: 20
Enter profit of item 2: 100
Enter weight of item 3: 30
Enter profit of item 3: 120
Maximum profit which could be obtained is: 240.00

Process returned 0 (0x0)   execution time : 22.103 s
Press ENTER to continue.

```

Lab Program 10:-

From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

```
#include<stdio.h>

#define MAX 10
#define INF 9999

int minDistance(int dist[],int visited[],int n)
{
    int min=INF,min_index;

    for(int v=0;v<n;v++)
        if(!visited[v]&&dist[v]<min)
        {
            min=dist[v];
            min_index=v;
        }

    return min_index;
}

void printSolution(int dist[],int n)
{
    printf("\nVertex\tDistance\n");

    for(int i=0;i<n;i++)
        printf("%d\t%d\n",i,dist[i]);
}
```

```

void dijkstra(int graph[MAX][MAX],int n,int src)
{
    int dist[MAX],visited[MAX];

    for(int i=0;i<n;i++)
    {
        dist[i]=INF;
        visited[i]=0;
    }

    dist[src]=0;

    for(int count=0;count<n-1;count++)
    {
        int u=minDistance(dist,visited,n);

        visited[u]=1;

        for(int v=0;v<n;v++)
            if(!visited[v]&&graph[u][v]&&dist[u]!=INF&&dist[u]+graph[u][v]<dist[v])
                dist[v]=dist[u]+graph[u][v];
    }

    printSolution(dist,n);
}

int main()
{

```

```

int n,src;

int graph[MAX][MAX];

printf("Enter number of vertices: ");
scanf("%d",&n);

printf("Enter adjacency matrix:\n");

for(int i=0;i<n;i++)
    for(int j=0;j<n;j++)
        scanf("%d",&graph[i][j]);

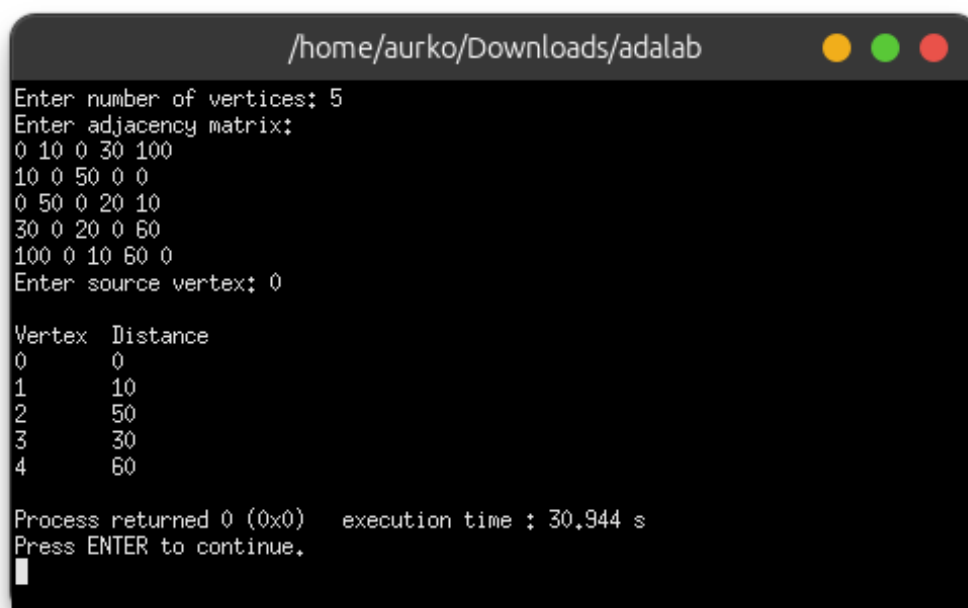
printf("Enter source vertex: ");
scanf("%d",&src);

dijkstra(graph,n,src);

return 0;
}

```

OUTPUT



```

/home/aurko/Downloads/adalab
Enter number of vertices: 5
Enter adjacency matrix:
0 10 0 30 100
10 0 50 0 0
0 50 0 20 10
30 0 20 0 60
100 0 10 60 0
Enter source vertex: 0

Vertex Distance
0          0
1          10
2          50
3          30
4          60

Process returned 0 (0x0)   execution time : 30.944 s
Press ENTER to continue.

```

Lab Program 11

Implement “N-Queens Problem” using Backtracking.

```
#include<stdio.h>

#define MAX 20

int board[MAX][MAX];
int N;

int isSafe(int row,int col)
{
    int i;

    for(i=0;i<row;i++)
        if(board[i][col]==1)
            return 0;

    for(i=1;row-i>=0&&col-i>=0;i++)
        if(board[row-i][col-i]==1)
            return 0;

    for(i=1;row-i>=0&&col+i<N;i++)
        if(board[row-i][col+i]==1)
            return 0;

    return 1;
}

int solveNQueens(int row)
```

```

{
    int col;

    if(row==N)
        return 1;

    for(col=0;col<N;col++)
    {
        if(isSafe(row,col))
        {
            board[row][col]=1;

            if(solveNQueens(row+1))
                return 1;

            board[row][col]=0;
        }
    }

    return 0;
}

void printBoard()
{
    int i,j;

    printf("\nSolution:\n\n");

    for(i=0;i<N;i++)

```

```

    {
        for(j=0;j<N;j++)
        {
            if(board[i][j]==1)
                printf("Q ");
            else
                printf(". ");
        }
        printf("\n");
    }
}

int main()
{
    int i,j;

    printf("Enter value of N: ");
    scanf("%d",&N);

    for(i=0;i<N;i++)
        for(j=0;j<N;j++)
            board[i][j]=0;

    if(solveNQueens(0))
        printBoard();
    else
        printf("No solution exists.\n");

    return 0;
}

```

}

OUTPUT

A terminal window with a dark background and light gray text. The title bar shows the path "/home/aurko/Downloads/adalab" and three window control buttons (yellow, green, red). The text inside the terminal reads: "Enter value of N: 4", "Solution:", followed by a 4x4 grid of dots and 'Q's representing a solution to the N-Queens problem. The grid is: Row 1: . Q . . ; Row 2: . . . Q ; Row 3: Q . . . ; Row 4: . . Q . . Below the grid, it says "Process returned 0 (0x0) execution time : 6.337 s" and "Press ENTER to continue." with a cursor on the next line.

```
/home/aurko/Downloads/adalab
Enter value of N: 4
Solution:
. Q . .
. . . Q
Q . . .
. . Q .

Process returned 0 (0x0)   execution time : 6.337 s
Press ENTER to continue.
█
```

LEETCODE PROBLEMS:-

Container with Most Water

```
int maxArea(int* height,int heightSize)
{
    int f=0,r=heightSize-1,maxa=0;

    while(f<r)
    {
        int h=height[f]<height[r]?height[f]:height[r];
        int a=h*(r-f);

        if(a>maxa)
            maxa=a;

        if(height[f]>height[r])
            r--;
        else
            f++;
    }

    return maxa;
}
```


Find First And Last Position of Element in Sorted Array

```
int* searchRange(int* nums,int numsSize,int target,int* returnSize)
```

```
{  
    int *ans=(int*)malloc(2*sizeof(int));  
    *returnSize=2;  
    ans[0]=-1;  
    ans[1]=-1;
```

```
    int l=0,r=numsSize-1;
```

```
    while(l<=r)
```

```
    {  
        int m=l+(r-l)/2;
```

```
        if(nums[m]>=target)r=m-1;
```

```
        else l=m+1;
```

```
    }
```

```
    if(l>=numsSize||nums[l]!=target)
```

```
        return ans;
```

```
    ans[0]=l;
```

```
    r=numsSize-1;
```

```
    while(l<=r)
```

```
    {  
        int m=l+(r-l)/2;
```

```
        if(nums[m]<=target)l=m+1;
```

```

        else r=m-1;

    }

    ans[1]=r;

    return ans;

}

```

The screenshot displays a coding platform interface with a dark theme. On the left, the problem description for "34. Find First and Last Position of Element in Sorted Array" is visible, marked as "Solved". It includes input/output examples and a runtime complexity requirement of $O(\log n)$. The right panel shows the submission status as "Accepted" with 88/88 testcases passed. A runtime graph indicates 0 ms execution time, beating 100.00% of other submissions. A user profile dropdown menu is open, showing the username "amitrakshar" and various navigation options like "My Lists", "Notebook", "Progress", "Points", "Try New Features", "Orders", "My Playgrounds", "Settings", "Appearance", and "Sign Out".

Course Schedule

```
bool canFinish(int numCourses,int** prerequisites,int prerequisitesSize,int*
prerequisitesColSize)
```

```
{
    int* indegree=(int*)calloc(numCourses,sizeof(int));
    int** graph=(int**)malloc(numCourses*sizeof(int*));
    int* graphSize=(int*)calloc(numCourses,sizeof(int));
```

```
    for(int i=0;i<numCourses;i++)
        graph[i]=(int*)malloc(prerequisitesSize*sizeof(int));
```

```
    for(int i=0;i<prerequisitesSize;i++)
```

```
    {
        int a=prerequisites[i][0];
        int b=prerequisites[i][1];

        graph[b][graphSize[b]++]=a;
        indegree[a]++;
    }
```

```
    int* queue=(int*)malloc(numCourses*sizeof(int));
    int front=0,rear=0;
```

```
    for(int i=0;i<numCourses;i++)
        if(indegree[i]==0)
            queue[rear++]=i;
```

```
    int count=0;
```

```
    while(front<rear)
```

```

{
    int node=queue[front++];
    count++;

    for(int i=0;i<graphSize[node];i++)
    {
        int next=graph[node][i];
        indegree[next]--;

        if(indegree[next]==0)
            queue[rear++]=next;
    }
}

return count==numCourses;
}

```

The screenshot shows the LeetCode interface for problem 207, "Course Schedule". The problem is marked as "Solved". The description states: "There are a total of `numCourses` courses you have to take, labeled from `0` to `numCourses - 1`. You are given an array `prerequisites` where `prerequisites[i] = [ai, bi]` indicates that you must take course `bi` first if you want to take course `ai`." It includes two examples: Example 1 with `numCourses = 2, prerequisites = [[1,0]]` resulting in `true`, and Example 2 with `numCourses = 2, prerequisites = [[1,0],[0,1]]` resulting in `false`. The submission interface on the right shows the code was "Accepted" with 54/54 testcases passed, a runtime of 43 ms, and a performance of 20.12%.

Find the duplicate number

```
int findDuplicate(int* nums,int numsSize)
{
    int slow=nums[0];
    int fast=nums[0];

    do
    {
        slow=nums[slow];
        fast=nums[nums[fast]];
    }
    while(slow!=fast);

    slow=nums[0];

    while(slow!=fast)
    {
        slow=nums[slow];
        fast=nums[fast];
    }

    return slow;
}
```

Problem List

287. Find the Duplicate Number

Solved

Medium

Topics

Companies

Given an array of integers `nums` containing $n + 1$ integers where each integer is in the range $[1, n]$ inclusive.

There is only one repeated number in `nums`, return this repeated number.

You must solve the problem without modifying the array `nums` and using only constant extra space.

Example 1:

Input: `nums = [1,3,4,2,2]`
Output: `2`

Example 2:

Input: `nums = [3,1,3,4,2]`
Output: `3`

Example 3:

Input: `nums = [3,3,3,3,3]`

25.6K 531 381 Online

Code

Accepted

All Submissions

Accepted 59 / 59 testcases passed

amitrakshar submitted at May 11, 2026 14:5

Runtime

5 ms Beats: 38.83%



Testcase Test Result

amitrakshar

Access all features with our Premium subscription!

My Lists

Notebook

Progress

Points

Try New Features

Orders

My Playgrounds

Settings

Appearance

Sign Out